

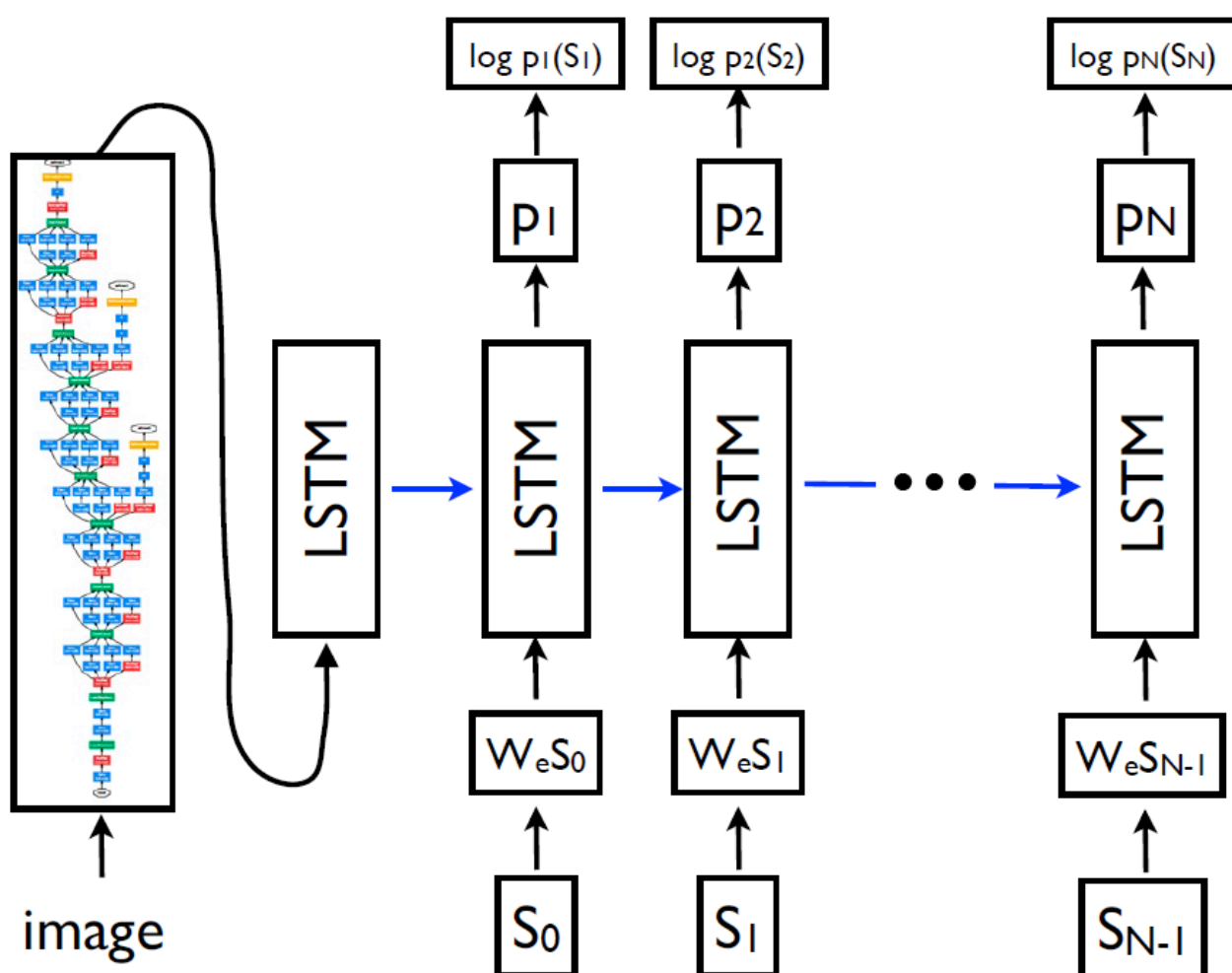
1. Описание задачи

Цель проекта — разработка модели, способной автоматически генерировать текстовые описания изображений.

2. Архитектура модели

В проекте используется архитектура типа encoder-decoder, состоящая из сверточной нейронной сети (CNN) и рекуррентной сети LSTM.

На рисунке ниже представлена общая схема модели.



Как показано на рисунке, модель состоит из двух частей: Encoder и Decoder.

Encoder (CNN) преобразует изображение в вектор признаков, который далее используется в качестве начального состояния для LSTM.

Decoder (LSTM) генерирует последовательность слов, используя предыдущие слова и внутреннее состояние сети.

2.1 Encoder (обработка изображения)

В качестве энкодера используется сверточная нейронная сеть (CNN), например ResNet.

Основная задача:

Преобразовать изображение в компактное векторное представление.

Процесс:

На вход подаётся изображение размерности:

$H \times W \times 3$

После прохождения через CNN извлекаются признаки:

image $\rightarrow$ feature vector (например, размерности 2048) Интерпретация:

Полученный вектор кодирует семантическую информацию:

объекты (например, собака, человек) сцена (улица, природа) контекст (движение, взаимодействие)

2.2 Decoder (генерация текста)

Декодер реализован с использованием LSTM (Long Short-Term Memory).

Основная задача:

Генерация последовательности слов по одному за шаг.

3. Механизм генерации текста

Генерация подписи происходит последовательно.

Шаги:

1. Начальное состояние LSTM инициализируется на основе вектора изображения
2. На вход подаётся специальный токен: <start
3. Далее модель предсказывает следующее слово

4. Обработка текстовых данных

Перед обучением текст проходит несколько этапов:

1. Токенизация. Предложения разбиваются на слова: "a dog runs"
2. Добавление специальных токенов <start a dog runs <end
3. Преобразование в числовой формат. Создаётся словарь : (<pad, <start, <end, a, dog) $\rightarrow$ (0, 1, 2, 3, 4)
4. Padding. Все последовательности приводятся к одной длине: [1, 3, 4, 2, 0, 0]

5. Предобработка изображений

Перед подачей в модель изображения проходят следующие преобразования:

5.1 Resize

Все изображения приводятся к фиксированному размеру (например, 224×224)

5.2 Нормализация

Значения пикселей переводятся в диапазон:

$[0, 255] \rightarrow [0, 1]$

6. Процесс обучения

Обучение модели происходит итеративно по эпохам.

6.1 Подготовка данных

Загрузка датасетов: Код получает изображения (из ClearML датасета "ImageDesc_dataset_v1") и аннотации (captions из "captions_vall"). Данные кэшируются локально, чтобы не скачивать повторно. Парсинг аннотаций: JSON-файл с captions разбирается, и captions группируются по ID изображений (image_id). Каждый ID имеет несколько описаний.

Построение словаря (Vocabulary): Из всех captions извлекаются слова (токенизация с NLTK). Создаётся словарь: слова $\rightarrow$ индексы (с специальными токенами: <pad, <start, <end, <unk). Слова, встречающиеся реже 5 раз, игнорируются. Токенизация captions: Каждый caption превращается в последовательность индексов слов (с <start в начале и <end в конце), дополняется <pad до длины 20.

Создание датасета и DataLoader: CaptionDataset загружает изображения (преобразует в тензоры, нормализует), пары их с captions. DataLoader разбивает данные на батчи по 32 примера (batch_size=32), перемешивает (shuffle=True).

6.2 Определение модели

EncoderCNN: Использует предобученный ResNet50 (без последнего слоя). Извлекает признаки из изображений (выход: вектор длины hidden_size=512). Параметры ResNet заморожены (не обучаются).

DecoderRNN: LSTM-сеть. Принимает признаки от encoder как начальное скрытое состояние. Генерирует последовательность слов: embedding $\rightarrow$ LSTM $\rightarrow$ linear слой для предсказания следующего слова.

CaptionModel: Комбинирует encoder и decoder. На вход: изображения и captions (для teacher forcing — модель видит правильные слова на предыдущих шагах).

6.3 Инициализация обучения

Модель на устройство: Перемещается на GPU (если доступно) или CPU.

Оптимизатор: Adam с lr=1e-4 — обновляет параметры модели.

Функция потерь: CrossEntropyLoss — измеряет ошибку предсказания слов (игнорирует).

Параметры: 5 эпох (num_epochs=5), ограничение на 10 батчей на эпоху (max_batches=10 для тестирования).

6.4 Цикл обучения

По эпохам (Epochs): 5 раз проход по всему датасету (или ограниченному количеству батчей).

По батчам (Batches): Для каждого батча (32 изображения + captions):

1. Forward Pass: Модель получает изображения и captions. Encoder извлекает признаки, decoder генерирует предсказания слов (выход: вероятности для каждого слова в словаре).
2. Вычисление Loss: Сравнивает предсказания с истинными словами (captions[:, 1:] — сдвиг на 1, чтобы предсказывать следующее слово). Loss = средняя ошибка по батчу.
3. Backward Pass: Вычисляет градиенты (loss.backward()).
4. Шаг оптимизатора: Обновляет веса модели (optimizer.step()), чтобы уменьшить loss.
5. Логирование: Loss за батч отправляется в ClearML (task.get_logger().report_scalar).
6. Конец эпохи: Вычисляется средний loss за эпоху (epoch_loss / len(dataloader)), логируется в ClearML.
7. Модель обучается только на decoder (encoder заморожен), чтобы адаптировать генерацию текста к признакам изображений.

6.5 Завершение и сохранение

График потерь: Строится график среднего loss по эпохам (matplotlib), сохраняется как PNG.

Сохранение модели: Веса модели (model.state_dict()) сохраняются в "caption_model_stage2.pth".

Логирование в ClearML: График и модель загружаются как артефакты (task.upload_artifact). Task закрывается.

Ключевые моменты:

1. Teacher Forcing: Модель видит правильные слова на входе, чтобы учиться быстрее.
2. Batch Processing: Обработка 32 примеров одновременно ускоряет обучение.
3. Ограничение: max_batches=10 — только 10 батчей на эпоху (для теста; для полного обучения установите len(dataloader)).
4. Время: Зависит от оборудования; на CPU ~несколько минут на эпоху, на GPU — быстрее.

7. Поведение функции потерь

В процессе обучения наблюдается уменьшение значения функции потерь.

Причина:

- модель постепенно улучшает предсказания
- увеличивается вероятность правильных слов
- уменьшается расхождение между предсказанием и истиной